

ENGINEERING RESEARCH REPORT

Harness

Architecture, Execution Model, AI Capabilities, and Competitive Positioning
for AI-Native Software Delivery

Prepared for: Enterprise Architects, Platform Engineers, DevSecOps Leads

Scope: Architecture · Pipeline/CI/CD Engine · AI Agents · Governance · Competitive Analysis

Research date: July 2026

How to read this report

This is a scoped, evidence-based cut through Harness's technical architecture and delivery process — not the full 23-section prompt. It prioritizes what an architect actually needs to make a build/adopt decision: how the system executes work, how governance is enforced, what the AI layer actually does today versus what's roadmap, and how it holds up against GitHub Actions, GitLab, Jenkins, ArgoCD, and Backstage. Each major claim is flagged:

DOCUMENTED confirmed in Harness's own architecture docs or verifiable product pages **ROADMAP** announced but not fully GA, or GA claim conflicts with vendor's own roadmap page **ANALYSIS** informed synthesis / industry framing, not a vendor claim

1. The Core Architectural Split: Control Plane vs. Delegate

Harness's entire execution model rests on one architectural decision: **the thing that decides what to run (control plane) is never the thing that touches your infrastructure (execution plane).** **DOCUMENTED**

Control Plane — Harness Manager: a multi-tenant cluster of microservices (SaaS or self-managed) that owns the pipeline engine, RBAC, secrets metadata, policy evaluation (OPA), UI, and task scheduling. It never executes customer workloads directly.

Data/Execution Plane — the Delegate: a customer-managed worker process that runs inside the customer's own network (Kubernetes, VM, or Docker host). The Delegate opens an **outbound** WebSocket (WSS) connection to the Manager on port 443 — no inbound ports are ever opened into the customer network. The Manager pushes task payloads down that socket; the Delegate executes against local infrastructure (cloud APIs, Kubernetes clusters, on-prem hosts) and streams results back.

This is the same pull-based trust model GitOps popularized with ArgoCD, generalized to every kind of task — builds, deployments, Terraform runs, security scans, chaos experiments — not just Kubernetes manifest sync. The practical consequence: Harness SaaS can sit outside your network entirely while still deploying into a PCI-segmented VPC, an air-gapped data center, or a regulated on-prem cluster, because the Delegate — not the control plane — is what needs network reachability.

Why this matters for a Fortune 100 estate

- **Zero inbound exposure:** security teams don't need to punch holes in firewalls for a SaaS vendor to reach production.
- **Heterogeneous trust boundaries:** different Delegates can be scoped to different clouds, business units, or compliance zones, each with its own credentials, while sharing one control plane, one RBAC model, and one audit trail.
- **Blast-radius containment:** Harness's own reference architecture explicitly recommends explicit-deny of raw CLI capability (aws-cli, kubectl) on Delegates and just-in-time access for anything requiring it, to contain the impact of a compromised or misused Delegate. **DOCUMENTED**

Delegate scaling patterns

Pattern	Description	Best fit
Centralized pool	One auto-scaled Delegate pool per Kubernetes cluster serves many applications/teams; build steps run as ephemeral K8s Jobs	CI workloads; teams that want central governance and platform-team ownership
BU-owned Delegates	Individual business units deploy and manage their own Delegate fleets	Orgs where BUs already own infra budgets and RBAC independently
Network-isolated Delegate	Delegate deployed inside a specific segmented boundary (PCI zone, air-gapped DC)	Regulated workloads that cannot share execution infrastructure with anything else

Source: Harness Delegate Architecture Best Practices reference doc. **DOCUMENTED**

CI execution specifically

For CI, the Delegate doesn't run the build itself — it spawns an ephemeral build pod containing a **Lite Engine** container that orchestrates step execution inside that pod (or, for VM-based build infrastructure, hands off to a Drone Runner managing dedicated build VMs). This ephemeral-pod-per-build model is architecturally similar to GitHub Actions' and GitLab's ephemeral runner model, but the pod is launched by infrastructure you control rather than a vendor-managed fleet.

2. Pipeline Engine and Governance Enforcement

Pipelines are defined as YAML and executed as a DAG by the Manager's pipeline engine, which supports parallel stages, conditional execution, templated/reusable step groups, approval gates, and rollback strategies. What differentiates Harness architecturally from a plain YAML-runner is that **policy evaluation is a first-class stage in the execution lifecycle, not a bolt-on check**.

Policy as Code (OPA)

Harness embeds Open Policy Agent as a central service and lets platform teams write Rego policies that are grouped into **Policy Sets**, each scoped to an entity type (Pipeline, Terraform Plan, Terraform State, IDP Catalog entity, Connector) and an event (On Save, On Run, On Step Start). Each policy carries a severity — *Error and Exit* or *Warn and Continue* — so governance can escalate from advisory to blocking without new tooling. **DOCUMENTED**

Example enforceable rule: "a pipeline cannot be saved with a production stage unless it contains an Approval step." This runs automatically on every save, at account, org, or project scope — meaning a central platform team can set an account-level policy that no project team can quietly bypass.

This same OPA engine also gates Infrastructure-as-Code Management (IaCM): policies can evaluate a Terraform *plan* or *state* file directly — catching a plan that would create an over-permissioned IAM role or an untagged resource before `apply` ever runs, rather than after.

RBAC model

Harness RBAC nests along **Account** → **Organization** → **Project**, matching how large enterprises are actually structured (Account for global/billing/SSO settings, Org typically for business units, Project for teams/applications). Roles + Resource Groups compose to form permission sets, and this same model extends into the Internal Developer Portal, so IDP self-service workflow execution rights are inherited from the underlying pipeline's RBAC rather than a separate permission system.

3. Deployment Strategies and Verification

CD strategies are implemented natively rather than as user-scripted logic:

Strategy	Mechanism	Primary tradeoff
Rolling	Sequential increments across nodes	Verification gates between nodes are harder to enforce mid-sequence
Blue-Green	Full parallel environment; traffic cutover via load balancer/Service selector	Fast, near-instant rollback; doubles infra cost during transition
Canary	Small traffic slice (e.g., 5%) expanded in phases against pass/fail metric criteria	Lower cost than blue-green; requires real traffic-shifting + observability integration
Kubernetes Canary	Harness generates a Canary group + a Primary group; Primary rollout uses native K8s RollingUpdate rather than manual phased traffic shifting	Simpler than classic canary because Kubernetes' own rolling update mechanics do the heavy lifting

Verification between phases can be AI-assisted: Harness compares live post-deploy metrics against a baseline window (commonly the pre-deploy period or a prior stable version) and can trigger automatic rollback if anomaly thresholds are breached — this is the modern form of what Harness originally branded "Continuous Verification," a capability the company has offered in some form since 2018, predating the current wave of "AI-native" branding. **DOCUMENTED**

4. Internal Developer Portal (Backstage-based, not Backstage)

Harness IDP is built *on top of* Backstage (uses the Backstage catalog model, supports its plugin ecosystem, and provides a Backstage-entity-YAML migration path) but is not self-hosted Backstage — it's a managed product layered with capabilities open-source Backstage doesn't ship with out of the box:

- **Entity-level granular RBAC** at a scale (tens of thousands of catalog entities) that Harness positions as beyond what unmodified Backstage handles gracefully for enterprises past a few hundred developers

- **OPA-based governance** applied directly to catalog entities (e.g., a service can't be promoted unless its IDP "scorecard" exceeds a threshold — enforced via the same Rego policy engine used for pipelines)
- **Self-service workflows that are literally Harness pipelines** — meaning a "request infrastructure" button in the developer portal inherits the exact same approval gates, audit trail, and RBAC as a production deployment pipeline, rather than being a separate scripted action
- **Fully managed SaaS** — no self-hosting, patching, or plugin-version maintenance burden

Source: Harness IDP vs. Backstage developer docs. **DOCUMENTED** — note this is vendor-authored comparison content; treat scale claims as Harness's framing, not independently audited benchmarks.

5. AI Capabilities: What's Actually Shipping vs. Roadmap

Harness's AI strategy is built around a shared **Software Delivery Knowledge Graph** — a connected map of services, pipelines, deployments, security findings, and cloud spend — that specialized agents reason over, rather than each agent operating on isolated context. As of mid-2026 the following are the major components:

Capability	What it does	Status
DevOps Agent / natural-language pipeline ops	Create/modify pipelines, troubleshoot failures, request infrastructure via natural language prompts against the Knowledge Graph	DOCUMENTED
AI QA Assistant	Generative, no-code test authoring; self-healing tests that adapt to UI changes; vendor claims ~10x faster test creation and ~70% less test-maintenance toil	DOCUMENTED (vendor benchmark, not independently verified)
SRE Agent + "Human-Aware Change Agent"	Automatic incident triage, postmortem generation; the Human-Aware variant uses "AI Scribe" to mine Slack/Teams/Zoom conversation for operational signal and correlate it against the Knowledge Graph's change data (e.g., linking a latency spike to a retry-config change 12 minutes earlier)	DOCUMENTED launch, ANALYSIS : meeting-content ingestion raises data-governance questions worth scoping explicitly in a rollout
AppSec agents	Generate security tests, detect vulnerabilities, propose fixes as inline code or PRs	DOCUMENTED
FinOps / cost agent	Cost Perspective rule generation, commitment analysis, Kubernetes spend-reduction recommendations	DOCUMENTED
Autonomous Worker Agents + Agent Marketplace	Any pipeline step can run as a reasoning agent (not fixed script) with sandboxed containers, scoped per-agent identity/credentials, and an LLM Gateway that policy-checks every model call. Six prebuilt agents (Autofix, Code Review, IaCM Remediation, etc.) ship out of the box; agents can call Anthropic, OpenAI, or Gemini models per pipeline. Also includes a Harness MCP Server so external tools (Claude Code, Cursor, Gemini CLI) can trigger Worker Agents.	Announced GA June 30, 2026 ROADMAP CAVEAT — independent coverage flags that Harness's own product page still lists "Agent Marketplace" as an H2 2026 roadmap item and the broader agents offering as "Limited Preview," which sits awkwardly against the GA announcement. Verify current status directly before planning around it.
Code Agent (IDE extension)	Standard IDE coding assistance, differentiated mainly by shared account context with the rest of the Harness platform	DOCUMENTED ; ANALYSIS — third-party reviewers note the IDE assistance itself is fairly standard vs. dedicated coding-agent products; the platform-context tie-in is the actual differentiator, not raw coding capability

Governance model for agentic execution: because Worker Agents run as pipeline-native steps, they inherit the same approval gates, RBAC, and audit trail as human-triggered deployments — an agent doesn't get to bypass a production approval gate just because it's an agent. Each agent has its own scoped identity/credential set rather than inheriting the invoking user's or pipeline's full permission set, and prompts/context pass through a policy-checking LLM Gateway before reaching a model. This is the practical answer to "how do we let AI touch production safely" that most CI/CD vendors are still bolting on as an afterthought.

DOCUMENTED

Early adopter signal (named, attributable): engineers at Verint Systems and United Airlines reported building a first production agent (Kubernetes-troubleshooting and security-remediation, respectively) in about four days each — a genuinely low barrier to a first agent, though scaling governance (who approves an agent for production, how per-agent audit/cost data feeds existing SRE/FinOps tooling) is the harder second step that hasn't been publicly demonstrated at large scale yet. **ANALYSIS**

6. Competitive Positioning

The most common RFP mistake, per multiple independent CI/CD comparison sources, is treating these as one category. They aren't: **CI runners** (GitHub Actions, GitLab CI, Jenkins, Harness CI) execute build/test jobs and produce artifacts; **CD operators** (ArgoCD) watch Git and reconcile cluster state; **Harness** and **GitLab** are unusual in trying to be both, plus governance, plus (in Harness's case) IDP and FinOps, under one control plane.

Dimension	Harness	GitHub Actions	GitLab	Jenkins	ArgoCD	Backstage
Core model	Managed control plane + self-hosted Delegate (push task, pull-connect)	Managed runners tied to GitHub repos	Integrated SCM+CI+security in one product; runner-based	Self-hosted master + agent plugins	Pull-based GitOps reconciler (K8s-native)	Open-source portal framework, self-hosted
Governance/ RBAC depth	Native OPA policy-as-code across pipelines, IaC, and IDP; Account/Org/Project nesting	Basic; relies on GitHub org permissions + third-party actions for policy	Strong, integrated (approvals, compliance frameworks, higher tiers)	Plugin-dependent; inconsistent without heavy customization	RBAC via K8s/Argo projects; not a general pipeline governance layer	Minimal out of the box; enterprises typically build custom RBAC
Vendor/tool lock-in	Works with any Git host; own execution layer	Tied to GitHub as SCM	Best when SCM + CI + registry all on GitLab	None (fully open) but heavy plugin lock-in over time	Kubernetes-only; needs a separate CI tool	None (framework, not a product)

AI depth (mid-2026)	Broadest: agentic pipeline steps, SRE/ AppSec/FinOps/ QA agents, shared knowledge graph, GA'd agent governance model	Copilot integration for code, limited pipeline-native agent tooling	GitLab Duo AI features across SDLC, growing but narrower agent-governance story than Harness's	Minimal native AI; relies on plugins	None natively (CD reconciler, not an AI product)	None natively
On-prem / air-gapped	Strong (Self-Managed Enterprise Edition, Delegate model built for network isolation)	Weak (GitHub Enterprise Server exists but is a different product)	Strong (mature self-managed option)	Best-in-class (fully self-hosted by design)	Good (native to K8s clusters, including air-gapped)	Good (self-hosted by definition)
Reported UX friction	Peer reviews note the UI feels "commercialized" vs. Argo/Jenkins; no pipeline-as-code parity with GitHub Actions/Jenkins; no true nested child-pipeline execution as of the review period	Simple, git-native YAML; large marketplace of 15,000+ actions	Mature, well-understood .gitlab-ci.yml model	High operational overhead maintaining plugins/infra	Steep learning curve for non-Kubernetes teams	Requires real engineering investment to operate at scale
Pricing model	Per-module commercial tiers; free tier exists across major products	Usage-based (minutes), generous free tier for public repos	Per-seat tiers (\$29–\$99/user/mo range reported)	Free/open-source; cost is operational overhead	Open-source (CNCF); cost is operational overhead	Open-source (CNCF); cost is operational overhead

Synthesized from independent comparison sources (Northflank, Opsio, JetBrains State of Developer Ecosystem, PeerSpot practitioner reviews) plus Harness's own comparison pages, which are flagged as vendor-authored where used. **ANALYSIS** for the synthesis; individual factual claims sourced as noted throughout.

Market share reality check

Despite Harness's platform breadth, adoption data tells a different story than the feature comparison: JetBrains' 2025/2026 developer ecosystem data puts GitHub Actions at roughly a third of organizational CI/CD adoption, Jenkins around 28%, and GitLab CI around 19% — Harness doesn't crack the top tier of

raw adoption, and PeerSpot engagement data shows both GitHub Actions' and Harness's "mindshare" metric declining year over year as of mid-2026. **DOCUMENTED** — this is a genuinely useful counterweight to vendor narrative: Harness's pitch is platform consolidation and governance depth for large regulated enterprises, not developer-mindshare dominance, and the adoption numbers are consistent with that positioning rather than contradicting it.

7. Decision Framework: When Does Harness Actually Make Sense?

Situation	Likely right call	Why
Small team, all-GitHub, simple build/test/deploy	GitHub Actions	Zero platform overhead; Harness's governance/IDP layer is dead weight at this scale
Single-vendor SDLC preference, mid-size org	GitLab	Comparable "one platform" philosophy with a more mature, larger install base and lower operational complexity than adding a second control plane
Heavy regulatory/compliance burden (bank, healthcare, gov), multi-BU, multi-cloud, need auditable policy enforcement across pipelines + IaC + AI agents under one governance model	Harness is a strong fit	This is the specific gap Harness is architected for: OPA-driven policy across every entity type, RBAC that mirrors enterprise org structure, and — as of 2026 — a governance model built for letting AI agents touch production safely
Already deep in Kubernetes/GitOps, want a pull-based CD reconciler specifically	ArgoCD (paired with any CI tool)	Harness CD can do this too, but if the org is Kubernetes-only and doesn't need the broader platform, ArgoCD is the CNCF-standard, lower-overhead choice
Heavy legacy Jenkins investment, air-gapped requirement, maximum customization need	Jenkins remains defensible	Full control and a mature plugin ecosystem still beat migration cost/risk in many regulated on-prem estates
Want an internal developer portal but don't want vendor lock-in or ongoing license cost, and have engineering capacity to own it	Backstage (self-hosted)	Full customization freedom; Harness IDP is the right call instead when the org lacks capacity to run/govern Backstage at scale or wants OPA/RBAC/audit built-in from day one

Framework synthesized from the comparison sources above. **ANALYSIS**

8. Key Risks and Diligence Items

- **GA-vs-roadmap ambiguity on the newest AI agent launch.** Independent reporting explicitly flags that Harness's Agent Marketplace GA announcement (June 30, 2026) conflicts with the company's own

product page, which still marks parts of the offering "Limited Preview" and lists the Marketplace as H2 2026. Confirm current status with Harness directly before committing a roadmap to it. **ROADMAP**

- **"Commercialized" UX friction and pipeline-as-code gaps** reported by practitioner reviewers (PeerSpot), including lack of nested child-pipeline execution as of the review period — verify against current release notes for your target version before assuming parity with GitHub Actions/Jenkins pipeline-as-code ergonomics.
- **Meeting-content ingestion for the Human-Aware Change Agent** (Slack/Teams/Zoom mining) is a genuine data-governance decision, not just a technical toggle — scope what gets captured, retained, and who can query it before enabling.
- **Vendor-authored comparison content** (Harness's own "Harness vs. X" pages) was used sparingly above and flagged; treat scale/performance claims sourced from Harness's own marketing (8x build speed, 80-to-1 effort reduction, etc.) as vendor benchmarks pending independent verification in your own environment.

9. Glossary

Term	Meaning
Control Plane / Harness Manager	Multi-tenant microservices cluster that owns pipeline engine, RBAC, policy, scheduling — never touches customer infra directly
Delegate	Customer-managed worker agent; opens outbound WSS to the Manager; executes tasks against local infra
Lite Engine	Container spawned inside ephemeral CI build pods to orchestrate step execution
Policy Set	A named group of OPA/Rego policies scoped to an entity type + event, with per-policy severity (Error and Exit / Warn and Continue)
Software Delivery Knowledge Graph	Harness's connected data layer spanning services, pipelines, deployments, security findings, and cost — the shared context AI agents reason over
Worker Agent	A pipeline step that runs as a reasoning LLM-backed agent instead of a fixed script, with sandboxing, scoped credentials, and policy-gated model calls via an LLM Gateway
IDP Scorecard	A computed score against a service in the Harness Internal Developer Portal, usable as an OPA policy input (e.g., gate promotion below a score threshold)

This report deliberately excludes company/funding history, org chart, and go-to-market material per scope adjustment during research. For the full 23-section prompt — including per-industry reference architectures, 15 hands-on labs, and sequence diagrams — use Claude's Research feature for a dedicated multi-hour pass; this document is intentionally the technical core rather than that full deliverable.